

BÁO CÁO TUẦN 3

1. Sơ đồ kiến trúc tổng thể hệ thống

1.1. Luồng dữ liệu tổng thể

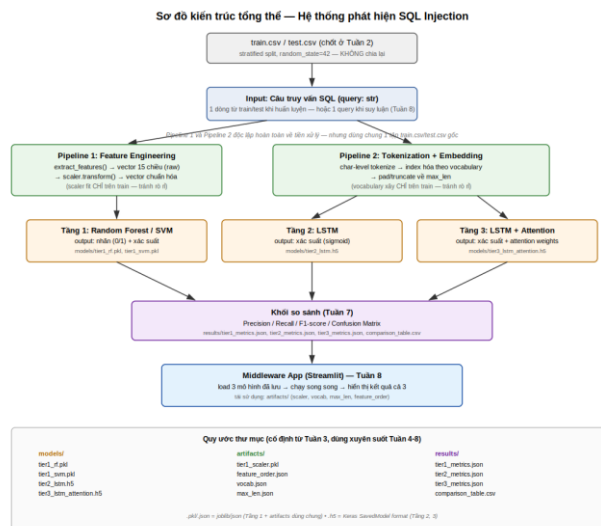
Hệ thống nhận đầu vào là một câu truy vấn SQL (query: str), có thể đến từ hai nguồn:

- Khi huấn luyện: từng dòng trong train.csv / test.csv (đã cố định ở Tuần 2, stratified split, random_state=42).
- Khi suy luận (Tuần 8): một câu truy vấn do người dùng nhập vào ứng dụng middleware.

Từ input, hệ thống rẽ thành hai pipeline tiền xử lý chạy song song và độc lập hoàn toàn với nhau:

- Pipeline 1 — Feature Engineering: phục vụ Tầng 1 (Random Forest/SVM).
- Pipeline 2 — Tokenization + Embedding: phục vụ Tầng 2 và Tầng 3 (LSTM, LSTM+Attention).

Kết quả dự đoán từ cả ba tầng được tổng hợp ở khối so sánh (Precision/Recall/F1/Confusion Matrix, thực hiện ở Tuần 7), và ba mô hình đã huấn luyện được tái sử dụng nguyên trạng trong ứng dụng middleware (Tuần 8).



Hình 1. Sơ đồ kiến trúc tổng thể hệ thống

1.2. Input/Output từng module

Module	Input	Output
Pipeline 1 – extract_features()	query: str	vector (16,) float, raw, thứ tự cột cố định
Pipeline 1 – scaler.transform()	vector (16,) raw	vector (16,) đã chuẩn hóa
Pipeline 2 – tokenizer	query: str	List[int] độ dài max_len (đã pad/truncate)
Tầng 1 (RF/SVM)	vector (16,) chuẩn hóa	label (0/1) + proba
Tầng 2 (LSTM)	sequence (max_len,)	proba (sigmoid)
Tầng 3 (LSTM+Attention)	sequence (max_len,)	proba + attention_weights (max_len,)
Khối so sánh	(y_true, y_pred, y_proba) của 3 tầng trên cùng test.csv	bảng metrics + confusion matrix mỗi tầng
Middleware	1 query từ người dùng	kết quả song song từ cả 3 tầng

1.3. Định dạng lưu mô hình

- Tầng 1: .pkl qua joblib — chuẩn phổ biến cho các object scikit-learn (RandomForest, SVC).
- Tầng 2, 3: .h5 — định dạng lưu chuẩn của Keras, gộp kiến trúc + trọng số trong 1 file, đơn giản hóa việc load lại ở middleware.
- Các đối tượng tiền xử lý (scaler, vocab, max_len) được lưu tách riêng khỏi mô hình, trong thư mục artifacts/, vì middleware cần load chúng trước bước đưa dữ liệu vào mô hình.

1.4. Quy ước thư mục (áp dụng xuyên suốt Tuần 4-8)

Khung project được tạo từ Tuần 1 gồm: app/, datasets/, docs/, models/, notebooks/, references/, results/, src/. Trong đó models/ và results/ đã có sẵn và được dùng đúng mục đích ban đầu (lưu mô hình, lưu kết quả đánh giá).

Bổ sung mới ở Tuần 3: thư mục artifacts/ (cấp gốc, ngang hàng models/) — không có trong khung project ban đầu, được thêm vào vì lý do kỹ thuật: các đối tượng tiền xử lý (scaler, vocabulary, max_len, thứ tự cột đặc trưng) không phải là mô hình và không phải là kết quả đánh giá, mà là trạng thái tiền xử lý bắt buộc phải tái sử dụng y nguyên giữa lúc huấn luyện và lúc suy luận (Tuần 8).

```
models/
├── tier1_rf.pkl
├── tier1_svm.pkl
```

```

├── tier2_lstm.h5
├── tier3_lstm_attention.h5

artifacts/
├── tier1_scaler.pkl          # MŨI – bổ sung từ Tuần 3, ngoài khung Tuần 1
                             # fit chỉ trên train, dùng .transform() cho
test/inference
├── clip_thresholds.json    # 12 ngưỡng percentile 99 (Pipeline 1)
├── feature_order.json      # thứ tự 16 cột cố định
├── vocab.json               # char -> index, 96 token, xây chỉ trên train
├── max_len.json

results/
├── tier1_metrics.json
├── tier2_metrics.json
├── tier3_metrics.json
├── comparison_table.csv

```

1.5. Ranh giới giữa hai pipeline

Pipeline 1 và Pipeline 2 không chia sẻ bất kỳ logic tiền xử lý nào — Pipeline 1 không dùng tokenizer, Pipeline 2 không dùng vector đặc trưng thủ công. Tuy nhiên cả hai cùng đọc từ một nguồn `train.csv/test.csv` gốc đã cố định ở Tuần 2, và cùng tuân thủ nguyên tắc: mọi thành phần fit trên dữ liệu (scaler, vocabulary) chỉ được fit trên `train.csv`, không bao giờ dùng `test.csv` để tránh rò rỉ thông tin (data leakage).

Đây là chủ đích thiết kế nhằm đảm bảo so sánh công bằng giữa hướng tiếp cận học máy cổ điển và học sâu trên cùng một bài toán, cùng một tập dữ liệu — cần nêu rõ điểm này khi bảo vệ đồ án để tránh bị hiểu nhầm là thiếu nhất quán giữa hai pipeline.

2. Đặc tả Pipeline 1 — Feature Engineering

2.1. Điều chỉnh so với budget đặc trưng ban đầu (Tuần 2)

Báo cáo Tuần 2 (mục 3) ghi tổng số đặc trưng đề xuất là 15, nhưng danh sách liệt kê chi tiết trong cùng mục chỉ có 14 tên cụ thể (4 cấu trúc + 6 ký tự đặc biệt + 4 từ khóa). Sau khi rà soát lại, quyết định chính thức cho Tuần 3:

- Giữ lại cả `n_semicolon` và `has_drop` (2 đặc trưng mà báo cáo Tuần 2, mục 3.4 đề xuất "cân nhắc loại bỏ") thay vì loại bỏ trước khi có bằng chứng thực nghiệm.
- Tổng số đặc trưng chính thức cho Tầng 1: 16 (không phải 15 như budget ban đầu) — thay đổi có chủ đích, ghi nhận lại để tránh hiểu nhầm khi đối chiếu ngược với báo cáo Tuần 2.
- Kế hoạch xử lý: sau khi huấn luyện Random Forest ở Tuần 4, xem xét feature importance của `n_semicolon`, `has_drop`, `keyword_repeat_count` và các `has_*`; nếu dùng SVM, bổ sung kiểm tra ma trận tương quan. Quyết định loại/gộp để dữ liệu quyết định ở Tuần 4.

2.2. Danh sách 16 đặc trưng — kiểu dữ liệu và công thức

#	Tên	Nhóm	Kiểu	Công thức / định nghĩa
0	length	Cấu trúc	int	Số ký tự của câu truy vấn
1	uppercase_ratio	Cấu trúc	float[0,1]	Số ký tự in hoa / length
2	n_whitespace	Cấu trúc	int	Số ký tự khoảng trắng
3	n_digit	Cấu trúc	int	Số ký tự chữ số (0-9)
4	n_quote	Ký tự đặc biệt	int	Số dấu '
5	n_equal	Ký tự đặc biệt	int	Số dấu =
6	n_comma	Ký tự đặc biệt	int	Số dấu ,
7	n_paren	Ký tự đặc biệt	int	Tổng số (và)
8	n_percent	Ký tự đặc biệt	int	Số dấu %
9	n_comment_markers	Ký tự đặc biệt	int	Tổng số --, #, /* */
10	keyword_repeat_count	Từ khóa	int	Tổng số lần từ khóa nguy hiểm lặp lại
11	n_semicolon	Bổ trợ	int	Số dấu ; — giữ lại, đánh giá lại Tuần 4
12	has_union	Từ khóa	binary	Có chứa UNION (không phân biệt hoa/thường)
13	has_or	Từ khóa	binary	Có chứa OR như một từ riêng
14	has_select_from	Từ khóa	binary	Có chứa cấu trúc SELECT...FROM
15	has_drop	Bổ trợ	binary	Có chứa DROP — giữ lại, đánh giá lại Tuần 4

Thứ tự cột 0-15 ở trên là thứ tự cố định, dùng xuyên suốt từ Tuần 4 đến Tuần 8, không thay đổi sau khi bắt đầu code.

2.3. Phương án xử lý outlier và scaling

Nguyên tắc phân nhóm: 12 cột đầu (index 0-11) là đặc trưng liên tục (đếm hoặc tỷ lệ) → cần xử lý outlier + scale. 4 cột cuối (index 12-15) là đặc trưng nhị phân → giữ nguyên, không xử lý.

Với 12 cột liên tục:

- Clip percentile 99, tính riêng cho từng cột (không dùng chung 1 ngưỡng cho cả 12 cột) — áp dụng đồng nhất cho toàn bộ nhóm vì nhiều đặc trưng ký tự đặc biệt có phân bố lệch/sparse tương tự length.
- StandardScaler, fit sau khi đã clip.

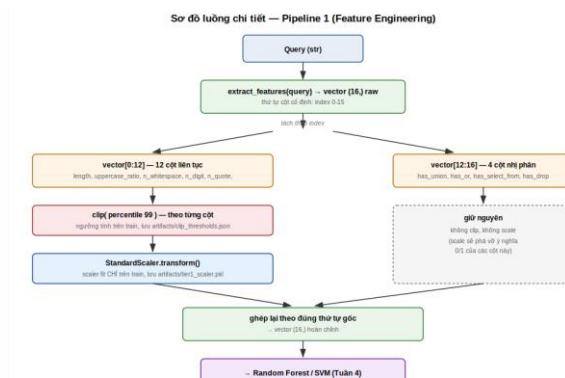
Lý do chọn clip thay vì RobustScaler: clip loại bỏ hoàn toàn khả năng outlier chi phối việc scale, còn RobustScaler tuy ít nhạy hơn nhưng vẫn giữ outlier trong dữ liệu, có thể ảnh hưởng đến cây quyết định của Random Forest.

Ràng buộc bắt buộc để tránh data leakage:

- Ngưỡng clip percentile 99 (12 giá trị, mỗi cột một ngưỡng) tính chỉ trên train.csv.
- StandardScaler fit chỉ trên train.csv (sau khi đã clip).

- Cả hai đều lưu lại thành artifact và tái sử dụng nguyên trạng cho test.csv và khi suy luận.

2.4. Sơ đồ luồng pipeline



Hình 2. Sơ đồ luồng chi tiết Pipeline 1

3. Đặc tả Pipeline 2 — Tokenization/Embedding

3.1. Quyết định: character-level tokenization

Tokenize theo từng ký tự, không tách theo từ. Lý do: payload SQLi thường dùng ký tự đặc biệt dày đặc (--, #, ', '(', ')...) không có ranh giới "từ" rõ ràng - Tuần 2 đã xác nhận nhóm ký tự đặc biệt là nhóm phân biệt mạnh nhất trong toàn bộ EDA (n_comment_markers chênh lệch ~820 lần).

3.2. Vocabulary

Số liệu thực tế tính trên train.csv (24.724 dòng, không dùng test.csv):

- Tổng số ký tự unique xuất hiện: 105
- 11 ký tự có tần suất ≤ 5 lần (control char, ký tự tiếng Hàn, dấu ngoặc kép cong...) — xác định là nhiễu dữ liệu
- 94 ký tự còn lại có tần suất ≥ 21 lần — có khoảng trống tự nhiên giữa tần suất 5 và 21

Ngưỡng lọc chính thức: giữ ký tự có tần suất ≥ 10 lần trong train.csv. Ký tự dưới ngưỡng và mọi ký tự chưa từng xuất hiện trong train đều gộp vào token <UNK>.

Vocab size chính thức: 96

- Index 0: <PAD> (đệm câu ngắn)
- Index 1: <UNK> (ký tự hiếm/không xác định)
- Index 2-95: 94 ký tự thật

Vocabulary được xây chỉ trên train.csv, không dùng test.csv, để tránh rò rỉ thông tin.

3.3. max_len

max_len = 250, chọn dựa trên phân bố độ dài thực tế trên toàn bộ train.csv (không tách theo nhãn):

Percentile	Giá trị
P90	154

Percentile	Giá trị
P95	221
P99	377
Max	5.370
Mean	68,92

250 nằm giữa P95 và P99 — phủ được nhiều hơn 95% nhưng ít hơn 99% số câu trong train mà không cần cắt. Có 1.082 dòng (~4,4% train) dài hơn 250 ký tự, trong đó 1.046/1.082 (96,7%) là nhãn SQLi.

3.4. Chiến lược truncating: head+tail

Phân tích vị trí từ khóa nguy hiểm trong 1.082 dòng dài hơn 250 ký tự cho thấy mâu thuẫn giữa các kỹ thuật tấn công:

Nhóm kỹ thuật	Vị trí trung bình (0=đầu, 1=cuối)	Số dòng
OR-based	0,06 (gần đầu)	464
UNION-based	0,71 (nghe nghiêng cuối)	86
Comment-based (--, #)	0,99 (gần như luôn ở cuối)	293

Không có phương án cắt một phía nào bảo toàn được cả hai nhóm vị trí đối lập. Do đó chọn head+tail truncation: với câu dài hơn 250 ký tự, giữ lại 125 ký tự đầu + 125 ký tự cuối, bỏ đoạn giữa.

Đánh đổi đã chấp nhận: phức tạp hơn khi code — cần viết hàm cắt tùy chỉnh, không dùng thẳng `keras.preprocessing.sequence.pad_sequences` cho phần truncate.

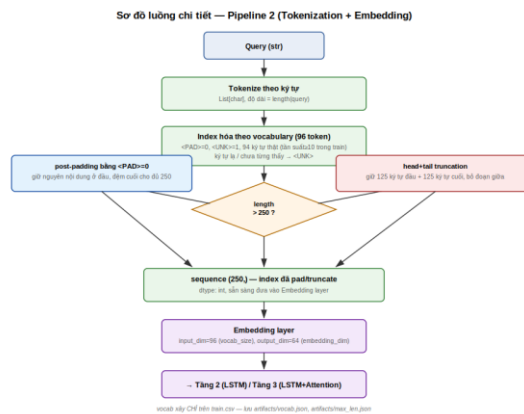
3.5. Padding

Khi $\text{length}(\text{query}) \leq 250$: giữ nguyên nội dung, post-padding bằng `<PAD>=0` vào cuối cho đủ 250 — mặc định của `pad_sequences`, không có mất thông tin ở bước này.

3.6. Embedding

`embedding_dim = 64` — điểm khởi đầu, tinh chỉnh thực nghiệm ở Tuần 5 nếu cần. Embedding layer: `input_dim=96 (vocab_size)`, `output_dim=64`.

3.7. Sơ đồ luồng pipeline



Hình 3. Sơ đồ luồng chi tiết Pipeline 2

3.8. Artifact cần lưu

- artifacts/vocab.json — mapping ký tự → index (96 token), xây chỉ trên train.
- artifacts/max_len.json — giá trị 250, cùng ghi chú chiến lược truncating (head+tail, 125/125).

4. Đặc tả kiến trúc chi tiết 3 mô hình

4.1. Bảng đặc tả tổng quan

Tầng	Input	Layers
1 — RF	vector(16,) chuẩn hóa	RandomForestClassifier(n_estimators=200, max_depth=None, class_weight='balanced')
1 — SVM	vector(16,) chuẩn hóa	SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced')
2 — LSTM	sequence(250,)	Embedding(96,64)→LSTM(64)→Dropout(0.2)→Dense(1,sigmoid)
3 — LSTM+Attn	sequence(250,)	Embedding(96,64)→LSTM(64,return_seq=True)→Attention→Dropout(0.2)→Dense(1,sigmoid)

4.2. Chi tiết Tầng 1

Random Forest:

- n_estimators=200 — chọn giữa khoảng gợi ý 100-300, cân bằng độ ổn định và thời gian huấn luyện.
- max_depth=None — không giới hạn ở bước khởi điểm, tuning Tuần 4 nếu overfit.
- class_weight='balanced' — chuẩn bị sẵn để xử lý mất cân bằng lớp 63/37, áp dụng chính thức Tuần 4.

SVM:

- kernel='rbf' làm baseline; C=1.0, gamma='scale' mặc định scikit-learn, tuning bằng GridSearch ở Tuần 4 nếu còn thời gian.

Đánh giá: cross-validation k=5 trên toàn bộ train.csv, không tách validation riêng — vì đây là baseline không cần theo dõi learning curve qua epoch.

4.3. Chi tiết Tầng 2 — LSTM baseline

```
Input: sequence (250,) int, giá trị index theo vocab (Pipeline 2)
→ Embedding(input_dim=96, output_dim=64)
→ LSTM(units=64, return_sequences=False)
→ Dropout(rate=0.2)
→ Dense(units=1, activation='sigmoid')
Output: xác suất thuộc lớp SQLi (0-1)
```

Không dùng Dense layer trung gian giữa LSTM và Output — quyết định có chủ đích để giữ mô hình đơn giản, giảm số tham số cần học.

4.4. Chi tiết Tầng 3 — LSTM + Attention

```
Input: sequence (250,) int – giống hệt Tầng 2
→ Embedding(input_dim=96, output_dim=64)           [giống Tầng 2]
→ LSTM(units=64, return_sequences=True)             [khác: return_sequences=True]
→ Attention layer (additive, tự cài đặt):
    score = Dense(1, activation='tanh')(lstm_output) # (batch,250,1)
    weights = softmax(score, axis=1)                 # (batch,250,1)
    context = sum(weights * lstm_output, axis=1)      # (batch,64)
→ Dropout(rate=0.2)                                  [giống Tầng 2]
→ Dense(units=1, activation='sigmoid')               [giống Tầng 2]
Output: xác suất (0-1) + attention_weights (250,)
```

Lý do chọn additive attention tự cài đặt thay vì tf.keras.layers.Attention có sẵn: mục tiêu Tuần 7 là trực quan hóa trọng số attention trên từng ký tự — additive attention cho ra trực tiếp 1 trọng số/1 vị trí thời gian, dễ vẽ heatmap.

Nguyên tắc bắt buộc: mọi thành phần khác ngoài Attention (Embedding, LSTM units, Dropout, Dense output, Loss, Optimizer) giữ nguyên vẹn giống Tầng 2 — đảm bảo khi so sánh kết quả ở Tuần 7, chênh lệch (nếu có) chỉ có thể quy cho việc bổ sung Attention.

4.5. Hyperparameter huấn luyện chung (Tầng 2 và Tầng 3)

Tham số	Giá trị	Ghi chú
Batch size	64	dùng chung cho cả 2 tầng
Epoch tối đa	30	dùng chung cho cả 2 tầng
Early stopping	val_loss, patience=5	dừng khi không cải thiện sau 5 epoch
Optimizer	Adam(lr=0.001)	dùng chung, không tinh chỉnh riêng

Việc cố định các siêu tham số này trước khi code Tuần 5-6 là điều kiện tiên quyết để kết luận so sánh ở Tuần 7 có giá trị khoa học.

4.6. Chỉ số đánh giá thống nhất

Áp dụng đồng nhất cho cả 3 tầng: Precision, Recall, F1-score, Confusion Matrix. Không dùng Accuracy làm chỉ số chính do lệch lớp (63/37).

5. Phương pháp chia Train/Validation/Test

5.1. Nguyên tắc chung

test.csv (6.181 dòng, cố định từ Tuần 2, random_state=42) giữ nguyên vai trò tập đánh giá cuối cùng, dùng chung cho cả 3 tầng, không bị động đến trong suốt quá trình thiết kế và huấn luyện.

train.csv (24.724 dòng) được chia tiếp thành train_split/val_split để phục vụ theo dõi overfitting khi huấn luyện Tầng 2, 3.

5.2. Kết quả chia thực tế

Chia theo tỷ lệ 80/20, có stratify theo nhãn, random_state=42:

Tập	Số dòng	Tỷ lệ lớp 0	Tỷ lệ lớp 1
train_split	19.779	63,18%	36,82%
val_split	4.945	63,20%	36,80%

Chênh lệch tỷ lệ nhãn giữa hai tập chỉ ~0,012%, xác nhận stratify hoạt động đúng.

Vị trí lưu: datasets/processed/train_split.csv, datasets/processed/val_split.csv — thư mục con mới, bổ sung vào datasets/ đã có sẵn từ khung project ban đầu.

5.3. Phương pháp áp dụng khác nhau giữa các tầng

Tầng 1 (Random Forest/SVM): dùng cross-validation k=5 trên toàn bộ train.csv. Lý do: đây là baseline không huấn luyện theo epoch, không cần theo dõi learning curve.

Tầng 2, 3 (LSTM, LSTM+Attention): dùng train_split để huấn luyện, val_split cố định để theo dõi val_loss qua từng epoch, làm cơ sở cho EarlyStopping(patience=5).

Lưu ý khi trình bày ở buổi bảo vệ đồ án: việc dùng hai phương pháp khác nhau là hợp lý về mặt kỹ thuật — xuất phát từ bản chất huấn luyện khác nhau giữa mô hình học máy cổ điển (không có khái niệm epoch) và mô hình học sâu (huấn luyện lặp qua nhiều epoch, cần tín hiệu dừng sớm). Đây không phải sự thiếu nhất quán, mà là lựa chọn phù hợp với đặc thù từng loại mô hình.

6. Nhận định rủi ro kỹ thuật khi bước sang thi công (Tuần 4-6)

6.1. Rủi ro liên quan đến Pipeline 1

- Bộ đặc trưng chính thức là 16, không phải 15 như budget ban đầu ở Tuần 2 — cần dùng feature importance (Random Forest) và ma trận tương quan (SVM) ở Tuần 4 để đánh giá thực chất vai trò của `n_semicolon`, `has_drop`, `keyword_repeat_count`. Nếu các đặc trưng này cho importance gần 0 hoặc tương quan cao với đặc trưng khác, cần gộp/loại và ghi lại lý do bằng số liệu, không loại theo cảm tính.
- Việc clip percentile 99 áp dụng đồng loạt cho cả 12 cột liên tục là quyết định thiết kế dựa trên quan sát định tính (phân bố lệch tương tự length); cần kiểm tra lại ở Tuần 4 xem việc clip có làm giảm khả năng phân biệt của đặc trưng nào không (ví dụ so sánh feature importance trước/sau khi thử bỏ clip cho một vài cột).

6.2. Rủi ro liên quan đến Pipeline 2

- Head+tail truncation phức tạp hơn khi code so với truncation một phía — cần viết hàm tùy chỉnh, không dùng thẳng `pad_sequences` cho phần truncate, tăng khả năng phát sinh lỗi off-by-one khi ghép lại 125 đầu + 125 cuối.
- Token <UNK>: tỷ lệ ký tự lạ trong train.csv rất thấp (11/105 ký tự, tần suất ≤ 5 lần), nhưng dữ liệu người dùng nhập thực tế ở Tuần 8 (middleware) có thể có phân bố ký tự khác hẳn train — cần theo dõi tỷ lệ <UNK> xuất hiện khi test thực tế, tỷ lệ quá cao đồng nghĩa vocab chưa đại diện đủ tốt.
- `max_len=250` chấp nhận cắt bớt thông tin ở $\sim 4,4\%$ dòng train (chủ yếu là SQLi) — cần theo dõi riêng hiệu năng phân loại trên nhóm câu dài này ở Tuần 5-6, không chỉ nhìn chỉ số tổng thể, để phát hiện sớm nếu head+tail truncation không đủ bảo toàn dấu hiệu tấn công.

6.3. Rủi ro liên quan đến kiến trúc mô hình

- Bắt buộc giữ nguyên hyperparameter (batch size=64, epoch tối đa=30, patience=5, Adam lr=0.001) giữa Tầng 2 và Tầng 3 khi code Tuần 5-6 — rủi ro lớn nhất là vô tình chỉnh riêng một tầng (ví dụ đổi batch size khi debug) mà quên đồng bộ lại, làm mất giá trị khoa học của kết luận so sánh ở Tuần 7.
- Additive attention tự cài đặt (không dùng layer có sẵn của Keras) cần được kiểm thử kỹ về mặt shape tensor trước khi huấn luyện chính thức — lỗi phổ biến khi tự viết attention là sai trục (axis) khi tính softmax, dẫn đến trọng số attention không tổng bằng 1 theo đúng chiều thời gian.

6.4. Rủi ro chung

- Outlier độ dài (max 5.370 ký tự) được xử lý theo hai cách khác nhau ở hai pipeline: clip percentile 99 ở Pipeline 1, `max_len` + head/tail truncation ở Pipeline 2 — cần diễn giải nhất quán trước hội đồng rằng đây là hai cơ chế xử lý phù hợp với bản chất dữ liệu đầu vào khác nhau của từng pipeline (vector đặc trưng số so với chuỗi ký tự), không phải sự tùy tiện.
- Thư mục artifacts/ là bổ sung mới ngoài khung project Tuần 1 — cần cập nhật lại README.md hoặc tài liệu mô tả cấu trúc project (nếu có) để phản ánh đúng thực tế, tránh gây nhầm lẫn khi có người khác (hoặc hội đồng) đối chiếu với mô tả ban đầu.